A presentation the agent can keep editing

Treat the deck as a version-controlled visual document—not a finished binary file.

Example deck · MVP

HTML · SVG · DATA · QA

The editable unit is the system

A deck becomes more useful when its source, evidence, design, and rendering can evolve together.

> The agent can change a claim, re-render the slide, and inspect the result without leaving the project.

source-001

ARCHITECTURE CHOICE

Keep PowerPoint at the edge, not at the core

PPT-FIRST

Binary as source of truth

- Harder to diff and review
- Manual updates hide data lineage
- Visual QA is mostly human

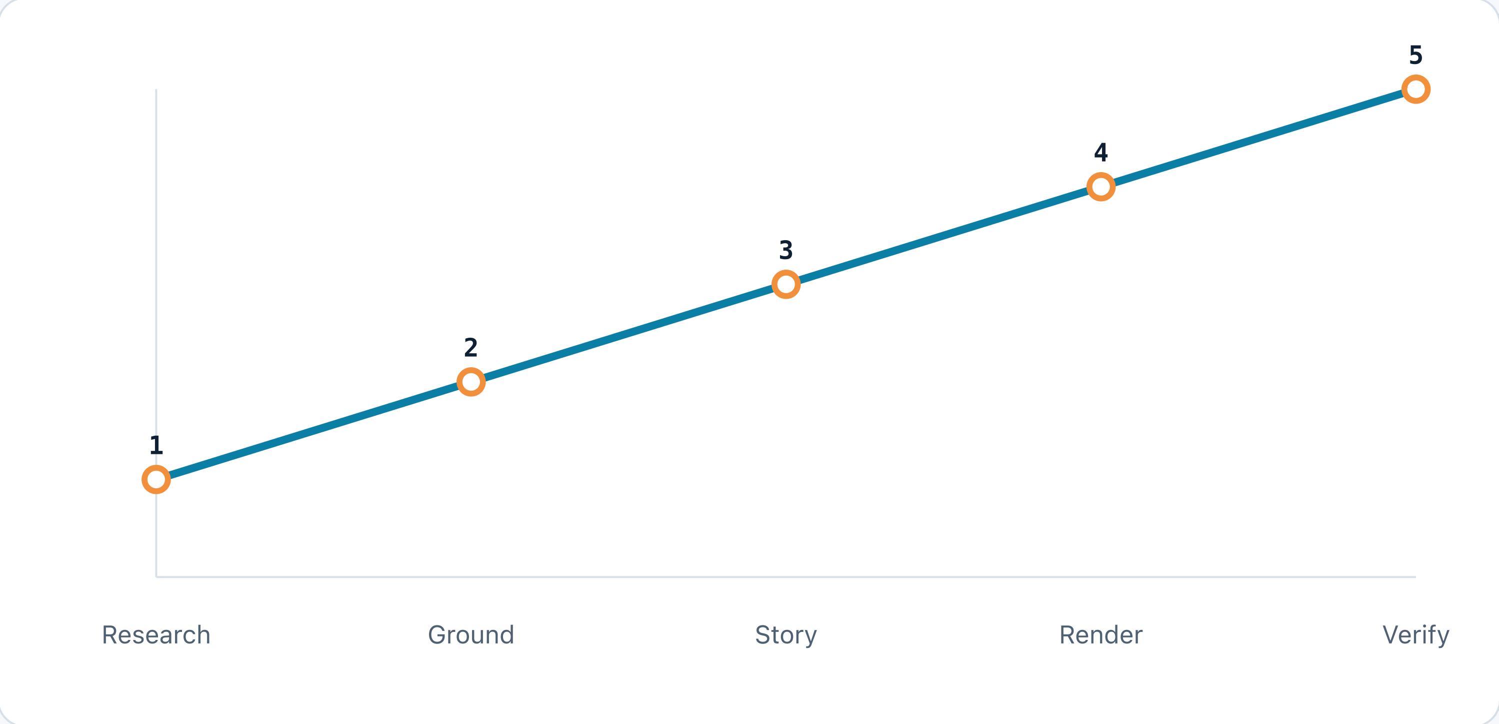
CODE-FIRST

HTML as source of truth

- Git can show every change
- Charts render from source data
- Agents can test and iterate

A DETERMINISTIC LOOP

The workflow makes each transformation explicit



WORKFLOW STAGES

A deck is a pipeline from source material to a checked visual artifact.

Ordinal stage values show the intended workflow sequence, not a performance benchmark.

source-001

AGENT WORKFLOW

Research should leave a trace at every handoff

01

Research

Collect files, repos, papers, data, and user context.

02

Ground

Turn observations into claims with source IDs.

03

Compose

Shape one primary message per slide.

04

Verify

Run layout, screenshot, grounding, and PDF checks.

source-001

MVP BUILD PATH

Start with a small engine, then earn complexity

Contract

Content + evidence

Make the authoring boundary legible to the next agent.

Runtime

Stage + navigation

Keep the design coordinate system fixed and predictable.

Visual

Components + SVG

Reuse primitives without forcing every slide into one layout.

QA

Checks + export

Make the artifact testable before calling it finished.

source-001 source-003

ITERATION POSTURE

Every visual change should be reversible

A local SVG asset keeps the idea inspectable and offline-safe.

`assets/agent-loop.svg`

THE AGENT LOOP

Render, inspect, fix

The screenshot is not the source. It is evidence that the source rendered as intended.

source-001 source-003

CODE PRIMITIVE

The interaction belongs in the engine, not the slide

engine/navigation.js JAVASCRIPT

```javascript
export function bindViewportNavigation(viewport, go) {
  let startX = 0;
  let drag = false;
  viewport.onpointerdown = (event) => { startX = event.clientX; drag = false; };
  viewport.onpointerup = (event) => {
    drag = Math.abs(event.clientX - startX) > 6 || Boolean(getSelection()?.toString());
  };
  viewport.onclick = (event) => {
    if (drag || event.target.closest('a,button,[data-no-nav]')) return;
    go(event.clientX < innerWidth / 2 ? -1 : 1);
  };
}
```

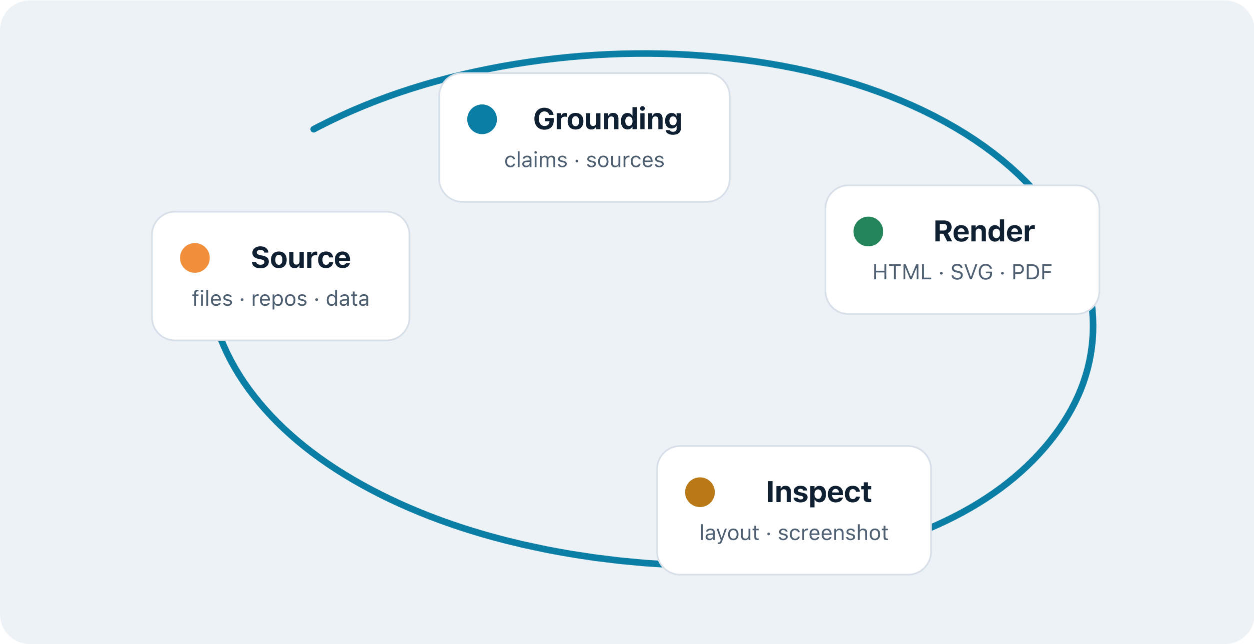
SOURCE PRIMITIVE

Keep slide content declarative; put behavior in reusable runtime code.

The example is intentionally short. The full runtime also guards links, buttons, selection, and print mode.

source-001 source-003

Animation should reveal the loop, not decorate it



A PURPOSEFUL SEQUENCE

Use motion to reveal causality, then leave a complete static state.

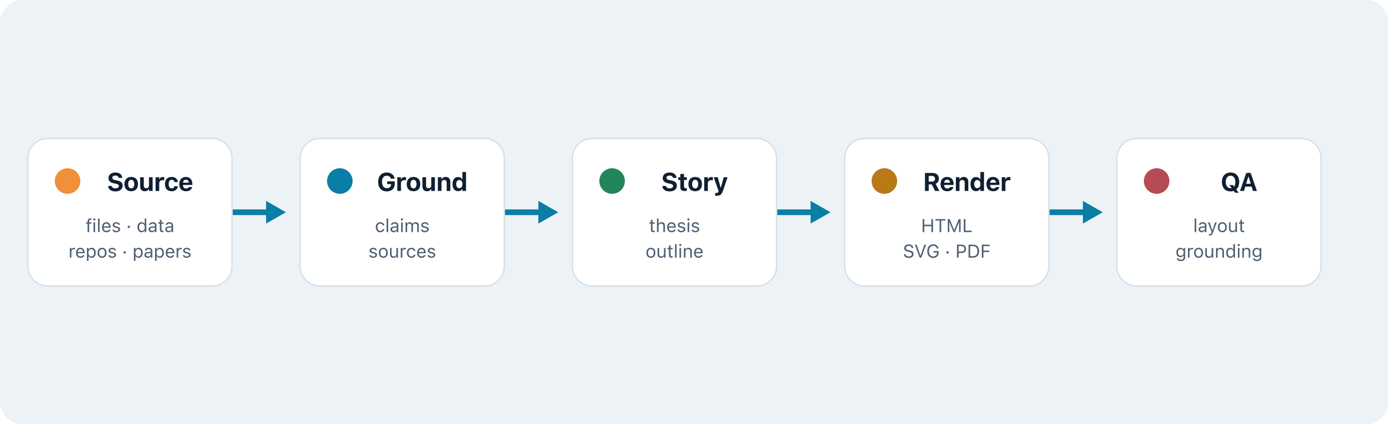
This loop draws the path first, then reveals the four stages. Print and reduced-motion modes show the final state.

Local SVG · reduced-motion safe · print safe

source-001 source-003

ONE-SHOT WORKFLOW

The agent turns messy inputs into a checked visual artifact



A SINGLE PASS

The animation ends where iteration begins.

Each stage appears once: source, grounding, story, render, then QA. Returning to this slide shows the final state without replaying.

Plays once on first entry · static after completion

source-001 source-003

A DESIGN HEURISTIC

Quality compounds when the system is explicit

```
Quality = Grounding × Determinism × Iteration
```

This is a design heuristic: remove any one factor and the deck becomes harder to trust or maintain.

source-001

Version-controlled. Grounded. Programmable.

The next deck should be another project that the agent can understand, change, and verify.